

Conv_Simple - Next 5 Tests (step by step)

You've flashed Conv_Simple_2.1 and captured the healthy 30 Hz run. These 5 fault tests come next - each fully spelled out. Work two terminals: A = the logger (capture), B = live_check / curl (confirm).

Before you start (once)

- CCW: DISCONNECT from the controller; PLC in RUN (so the Modbus server answers).
- Historian up + connected: `curl http://127.0.0.1:8766/health -> "connection":"ok"`.
- Two terminals open in the repo root. (Optional) grab `baseline_idle` with the logger for reference.
- Read-only: the logger + live_check only READ Modbus - nothing writes to the PLC or GS10.
- Keep a hand near the e-stop; only manipulate e-stop wiring with the rig DE-ENERGIZED.

Test 1 - PLC / bridge offline

A0 CRITICAL

Goal: Prove MIRA notices when the PLC stops talking to the historian.

1. Terminal A - start a labeled capture:

```
python plc/conv_simple_anomaly/live_logger.py --label a0_offline
```

2. Unplug the PLC Ethernet cable (from the laptop or the PLC).

3. Wait ~35 seconds - A0 needs ≥ 30 s of no fresh data.

4. Terminal B - confirm the detector sees it:

```
curl http://127.0.0.1:8766/health
```

-> expect: "connection":"offline" (no fresh data = the A0 condition). live_check can't run with the link down - that absence IS the fault.

5. Plug Ethernet back in; wait ~5 s.

```
curl http://127.0.0.1:8766/health
```

-> expect: "connection":"ok" - recovered.

6. Terminal A - stop the logger (Ctrl-C). Log saved under logs/.

7. Ask MIRA: "Is the conveyor PLC online? It went quiet."

-> expect: "Lost contact with the PLC - no fresh data for N s. Check the Ethernet link to 192.168.1.100 and that the bridge/historian is running."

Test 2 - GS10 RS-485 link down

A1 + B5 CRITICAL

Goal: Prove MIRA flags STALE drive data when the GS10 serial link drops (Ethernet stays up).

1. Terminal A - capture:

```
python plc/conv_simple_anomaly/live_logger.py --label a1_rs485_down
```

2. Unplug the RS-485 lead between the PLC and the GS10. Leave Ethernet connected.

3. Wait ~6 s.

4. Terminal B - confirm the rule fires:

```
python plc/conv_simple_anomaly/live_check.py --secs 6
```

-> expect: [CRITICAL] A1_COMM_STALE ... and in the snapshot `vfd/vfd101/comm_ok = False`.

note: B5: the VFD registers still show their LAST values (frozen, not fresh) - the trust-gate marks them stale so MIRA won't reason on dead numbers.

5. Reconnect the RS-485 lead; wait ~5 s.

```
python plc/conv_simple_anomaly/live_check.py --secs 6
```

-> expect: A1 clears (`comm_ok = True`).

note: If the GS10 faulted on the comm loss, this is also Test 5 (A2 / CE10) - expected; reset the drive after.

6. Terminal A - Ctrl-C. Ask MIRA: "Is the GS10 talking? Why are its values frozen?"

-> expect: "GS10 isn't answering on RS-485 - values are stale. Likely a loose/reseated conductor or termination; check the A/B pair, confirm 9600 8N1 slave 1."

Test 3 - E-stop: normal press, then wiring fault

S1 + A3 HIGH

Goal: Prove MIRA tells a normal e-stop press apart from a dangerous wiring fault.

1. Terminal A - capture:

```
python plc/conv_simple_anomaly/live_logger.py --label estop
```

2. WARM-UP (S1, safe): press the E-stop mushroom.

```
python plc/conv_simple_anomaly/live_check.py --secs 4
```

-> expect: snapshot shows safety/estop = True and NO anomaly - a healthy intentional stop.

3. Release the E-stop and re-arm (Start).

4. FAULT (A3): with the motor DE-ENERGIZED, pull ONE e-stop channel wire (NC or NO) so the two channels disagree.

```
python plc/conv_simple_anomaly/live_check.py --secs 6
```

-> expect: [HIGH] A3_ESTOP_WIRING ... with di02_estop_nc == di03_estop_no (both read same).

5. Restore the wire; re-check that A3 clears:

```
python plc/conv_simple_anomaly/live_check.py --secs 4
```

6. Terminal A - Ctrl-C. Ask MIRA: "Is there an e-stop wiring fault?"

-> expect: "E-stop channels disagree (NC vs NO) - one contact or wire has failed; this can defeat the stop. Verify both channels switch together; check the suspect leg."

note: Safety: this touches a safety circuit. De-energize first; restore the wire before returning the rig to service. Press (warm-up) freely; pull a wire only de-energized.

Test 4 - Direction conflict (both selected)

A4 MEDIUM

Goal: Prove MIRA explains why the belt won't move when both directions are commanded.

1. Terminal A - capture:

```
python plc/conv_simple_anomaly/live_logger.py --label a4_direction
```

2. Throw BOTH direction switches (FWD and REV) at once.

```
python plc/conv_simple_anomaly/live_check.py --secs 6
```

-> expect: [MEDIUM] A4_DIRECTION_FAULT ... with di00_fwd & di01_rev both True, cmd_word = 1 (STOP).

3. Return to a single direction (or neutral).

4. Terminal A - Ctrl-C. Ask MIRA: "Why won't the belt pick a direction?"

-> expect: "Both FWD and REV are selected - the PLC won't choose, so it commands STOP. Check the selector/wiring; only one should be made at a time."

Test 5 - GS10 drive fault decode (the showcase)

A2 HIGH

Goal: Prove MIRA decodes a real GS10 fault CODE into a plain-language cause + fix. (Newly unblocked by today's V2.1 flash.)

1. Terminal A - capture:

```
python plc/conv_simple_anomaly/live_logger.py --label a2_ce10
```

2. Unplug the RS-485 for > 5 s (past the GS10 P09.03 comm timeout). The drive should trip CE10 if P09.02 is set to fault-on-comm-loss.

3. Reconnect the RS-485; wait ~5 s for the PLC to re-poll the drive.

```
python plc/conv_simple_anomaly/live_check.py --secs 6
```

-> expect: [HIGH] A2_VFD_FAULT: GS10 reports fault code 58: CE10 (Modbus timeout) ... with vfd/vfd101/fault_code = 58.

note: If fault_code stays 0, the drive rode through (P09.02 not set to fault) - no trip to decode. Alternative hard fault: oL (overload) - but that needs a real belt load (the unloaded bench draws ~0.5 A, well under the oL threshold).

4. Reset the GS10 fault (keypad STOP/RESET, or power-cycle the drive). Re-check it clears:

```
python plc/conv_simple_anomaly/live_check.py --secs 4
```

5. Terminal A - Ctrl-C. Ask MIRA: "What fault is the GS10 showing?"

-> expect: "GS10 tripped: CE10 (Modbus timeout) - lost the RS-485 link long enough to fault. Restore comms, clear the cause, reset the drive; check P09.02 if you don't want a hard trip on brief comm loss."

After the 5

You'll have 5 labeled logs under logs/ - the ground truth for each fault. Those become the replay fixtures + golden cases in Phase 1 of the proving plan (docs/plans/2026-06-14-proving-test-case-plan.md). Tell me when they're captured and I'll turn them into the regression suite. Deferred for later: load-dependent faults (A8 overcurrent, A7 freq-tracking) need a belt load; the photo-eye family (A12, E1-E7) needs the di05 reflash.